MARKSMITH

CS23531 – WEB PROGRAMMING
MINI PROJECT REPORT

Submitted by

KAILASH BALASUNDARATHURAI - 241001101

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**MARKSMITH – A HANDCRAFTED MARKDOWN STUDIO**" is the Bonafide work of **KAILASH BALASUNDARATHURAI (241001101)** who carried out the work under my supervision.

SIGNATURE
Dr.P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

SIGNATURE
Ms. Janani M
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

This project is submitted for CS23531 – Web Programming held on _____

INTERNAL EXAMINAR **EXTERNAL EXAMINAR**

MARKSMITH

ABSTRACT:

Marksmith is a fast, responsive Markdown previewer and editing studio built entirely with HTML5, CSS3 and Vanilla JavaScript (ES6+). It provides a split-screen workspace in which the Markdown typed in the editor pane is parsed and rendered instantly in the preview pane, without any page reload. The editor offers tactile writing features such as smart indentation, automatic list continuation, bracket and quote auto-pairing, and a formatting toolbar for headings, bold, italic, code, quotes, tables, lists, task checkboxes and links. The preview supports GitHub-flavoured Markdown, syntax-highlighted code blocks with a one-click copy button, GitHub-style alert callouts, interactive task checklists and LaTeX mathematics. Users can switch between Split, Write and Read modes, drag the divider to resize the panes, choose between several colour themes and preview typefaces, import .md files by drag and drop, and export their work as Markdown, standalone HTML or a print-ready PDF. All work is saved automatically to localStorage. The only external libraries used are marked.js, Highlight.js and KaTeX, loaded from CDNs. The design follows a utilitarian, writer-focused aesthetic inspired by Obsidian, Typora, iA Writer and GitHub.

- KAILASH BALASUNDARATHURAI

TABLE OF CONTENTS

S.No	Title	Page No
1.1	Introduction	5
1.2	Purpose	5
1.3	Scope of the project	5
1.4	Software requirement specification	7
2.1	Use case diagram	9
2.2	System architecture diagram	10
2.3	Data flow diagram	11
3	Module description	12
4	Implementation	13
5	Conclusion	17
6	References	18

MARKSMITH

1.1 INTRODUCTION

Markdown is one of the most widely used lightweight markup languages. It is the standard format for README files, technical documentation, notes, blog posts and developer wikis. Writing it comfortably requires a previewer that shows the formatted result as the text is typed. Many online Markdown tools are cluttered with advertisements, require an account, or send the user's text to a remote server. Marksmith is a web application that reimagines the Markdown previewer as a fast, private and tactile writing studio. The editor and the rendered output sit side by side, the preview updates on every keystroke, and all processing happens inside the browser. Users can write, format, preview, save and export documents without any page reloads.

1.2 PURPOSE

The purpose of Marksmith is to give students, developers and writers a fast, distraction-free and accessible environment for writing Markdown with an accurate live preview. It aims to:

- Display the rendered output beside the editor and refresh it instantly on every edit without reloading the page.
- Speed up writing with smart editing features: automatic indentation, list continuation, bracket and quote auto-pairing, a formatting toolbar and keyboard shortcuts.
- Render GitHub-flavoured Markdown faithfully, including tables, task lists, alert callouts, syntax-highlighted code blocks and mathematical formulas.
- Provide a flexible workspace with Split, Write and Read modes, a draggable divider and synchronised scrolling.
- Preserve the user's work and theme preference across sessions using localStorage auto-save.
- Support easy import and export through file open, drag and drop, Markdown download, standalone HTML export, Copy HTML and print to PDF.

1.3 SCOPE OF THE PROJECT

Marksmith is a client-side, single-page web application intended for writing, studying and documentation use. Its scope covers:

- **Markdown Editor:** A textarea-based editor with a live cursor-position indicator, smart Tab and Shift+Tab indentation, automatic list and checklist continuation, bracket and quote auto-pairing and pair backspacing.
- **Live Rendering:** The editor text is parsed by marked.js on every input event and displayed in the preview pane, with Highlight.js syntax colouring for code blocks and KaTeX for inline and display maths.
- **Formatting Toolbar:** One-click buttons for headings, bold, italic, strikethrough, inline code, code blocks, quotes, tables, bullet and numbered lists, task checkboxes and links.

- **Workspace Layout:** Three view modes (Split, Write, Read), a draggable divider that resizes the panes between 15% and 85%, and proportional synchronised scrolling that can be switched off.
- **Themes and Typography:** Multiple colour themes (including a light and a dark theme) and a choice of sans-serif, serif or monospace typography for the rendered preview.
- **Interactive Checklists:** Task checkboxes clicked in the preview update the raw Markdown in the editor automatically.
- **File Handling and Export:** Open and drag-and-drop import of .md, .markdown and .txt files, Export .md, standalone HTML export, Copy HTML and a print-ready PDF layout.
- **Document Statistics and Auto-Save:** A status bar showing word, character and line counts and estimated reading time, with continuous auto-save to localStorage.
- **Preset Templates:** Starter documents such as a technical README, an essay, a sprint sync note and a full Markdown syntax guide.

The project does not include a backend, database, user accounts or server-side logic. It runs entirely in the browser from static files.

1.4 SOFTWARE REQUIREMENTS OF THE PROJECT

Overall Description

Marksmith runs entirely in the browser as a static web application. There is no server-side logic or database. The project files (index.html, the CSS files and the JavaScript files) can be opened directly in any modern browser or served from any static web host.

Product Perspective

The system is a single-tier, client-only application:

- **Presentation tier:** index.html defines the semantic structure of the header, toolbar, editor pane, preview pane and status bar; style.css defines the layout, split-pane system, theme tokens and print styles; markdown.css styles the rendered Markdown output.
- **Logic tier:** app.js controls the application state, divider resizing, scroll synchronisation and export; editor.js handles tabs, auto-pairing, shortcuts and list logic; renderer.js handles parsing, highlighting, alerts and maths; templates.js holds the starter documents.

Hardware Requirements

- Processor: Any modern CPU
- RAM: 2 GB or higher
- Any device with a modern web browser

Software Requirements

- Browser: Google Chrome, Mozilla Firefox, Microsoft Edge or Safari (any modern version)
- No backend, server or database required
- No npm, build tools or frameworks required

Product Functionality

- **Live Preview:** Parses the editor text and renders it in the preview pane on every keystroke.
- **Formatting Toolbar:** Wraps the selected text or inserts Markdown syntax at the cursor with one click.
- **Smart Editing:** Indents with Tab, continues lists with Enter and auto-pairs brackets and quotes.
- **View Modes:** Switches the workspace between Split, Write and Read layouts.
- **Resizable Panes:** Dragging the divider changes the pane widths; double-clicking resets it to 50/50.
- **Synchronised Scrolling:** Keeps the editor and preview scroll positions proportional.
- **Theme Toggle:** Switches themes and saves the preference in localStorage.

- **File Controls:** Open File, drag-and-drop import, Export .md, Copy HTML and Clear.

Assumptions and Dependencies

- A modern browser with ES6+ JavaScript support is available.
- marked.js and Highlight.js (and KaTeX for maths) are loaded from public CDNs (jsDelivr and cdnjs); an internet connection is needed on the first visit to download them.
- JavaScript and localStorage must be enabled in the browser.

Functional Requirements

Editor Module (EM)

- Pressing Tab inserts two spaces; Shift+Tab unindents a single line or a multi-line selection.
- Pressing Enter on a bullet, numbered or task list line starts the next list item automatically; Enter on an empty item removes the marker.
- Typing (, [, { or ` inserts the closing character; if text is selected it is wrapped instead.
- The cursor position (line and column) is updated in the editor pane bar as the cursor moves.

Rendering Module (RM)

- On every input event, the content of #markdown-input is parsed by marked.js and written into #preview-output.
- Code blocks are highlighted by Highlight.js and receive a language label and a Copy Code button.
- Blockquotes starting with [!NOTE], [!TIP], [!IMPORTANT], [!WARNING] or [!CAUTION] are rendered as coloured callouts; $...$ and $$...$$ are rendered with KaTeX.
- Clicking a task checkbox in the preview toggles the matching [ ] or [x] in the editor text.

Toolbar Module (TM)

- Each toolbar button carries a data-action value (h1, h2, bold, italic, strike, code, codeblock, quote, table, ul, ol, task, link).
- Clicking a button wraps the current selection or inserts a Markdown template at the cursor.
- Keyboard shortcuts: Ctrl+B, Ctrl+I, Ctrl+K, Ctrl+Shift+X, Ctrl+S and Ctrl+/.

View and Layout Module (VLM)

- The Split, Write and Read buttons carry a data-view value and change the mode class of the #workspace element.
- The resizer is draggable between 15% and 85% and resets to 50/50 on double-click.
- The Sync Scroll checkbox switches proportional scroll synchronisation on or off.

File and Export Module (FEM)

- Open File and drag-and-drop read .md, .markdown and .txt files with the FileReader API into the editor.

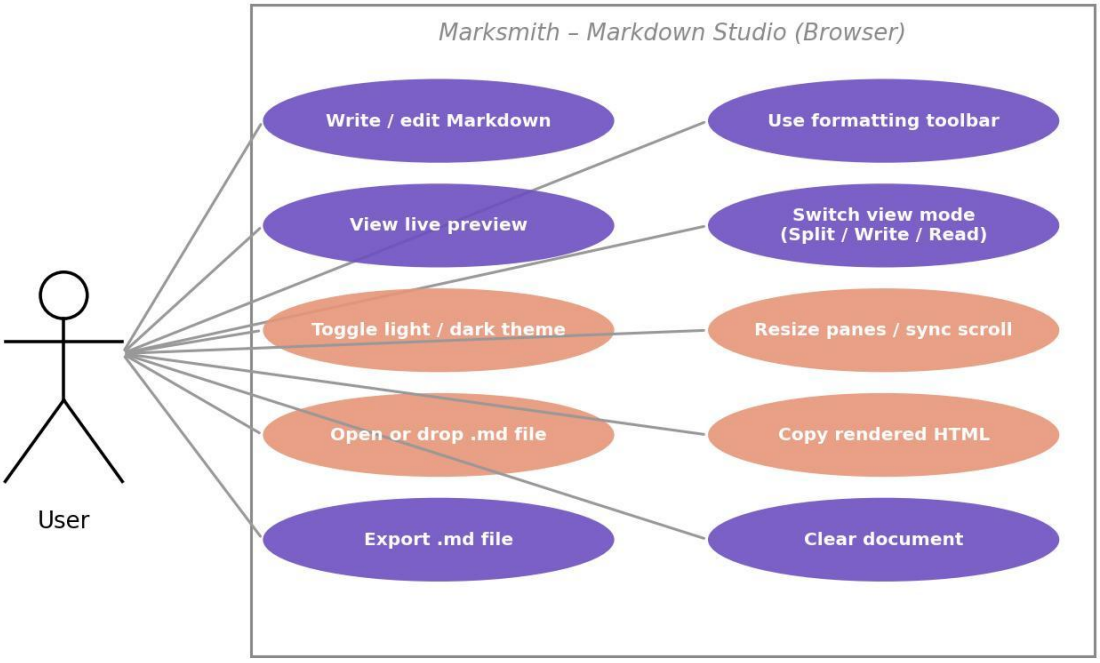
- Export .md downloads the text as a file named from the first H1 title; HTML export produces a standalone styled page.
- Copy HTML copies the rendered HTML to the clipboard; Clear empties the editor.

Theme and Storage Module (TSM)

- The editor content is saved to localStorage automatically and the status bar shows the save state.
- The theme button changes the data-theme attribute on the html element and the choice is stored in localStorage.

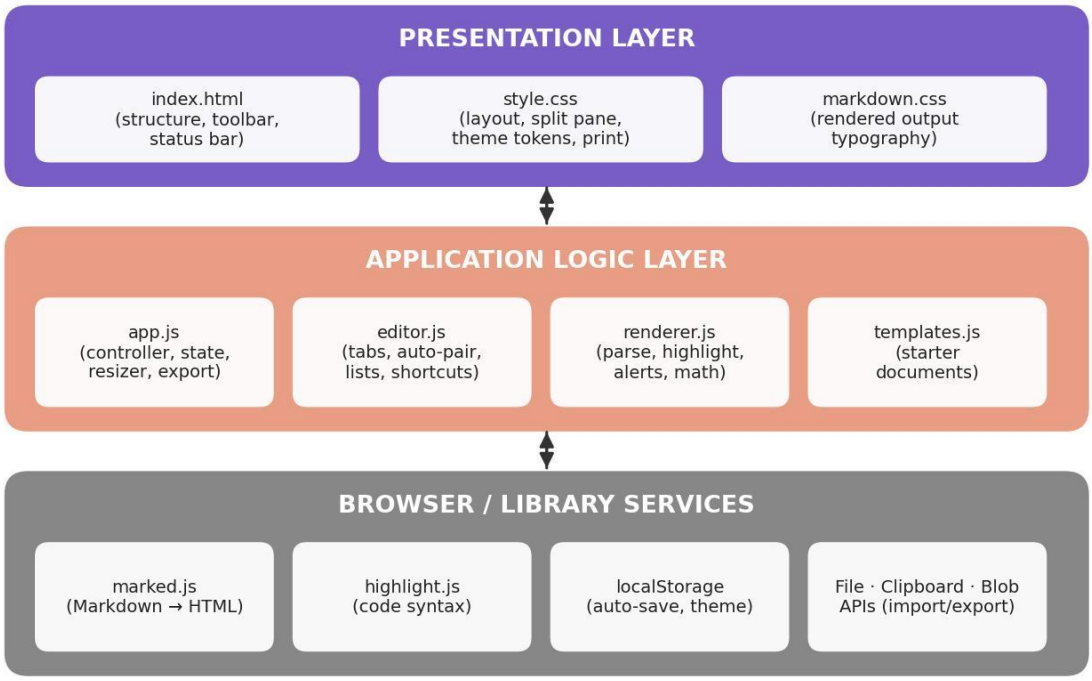
2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAM



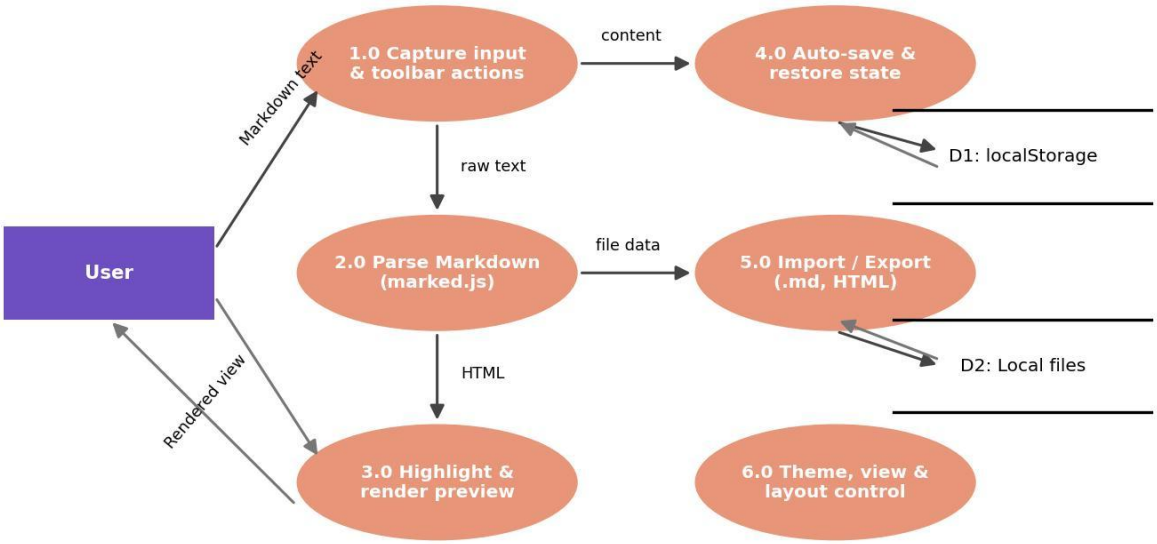
2.2 SYSTEM ARCHITECTURE DIAGRAM

Single-tier, client-only architecture (runs fully in the browser)



2.3 DATA FLOW DIAGRAM

Level-1 Data Flow Diagram

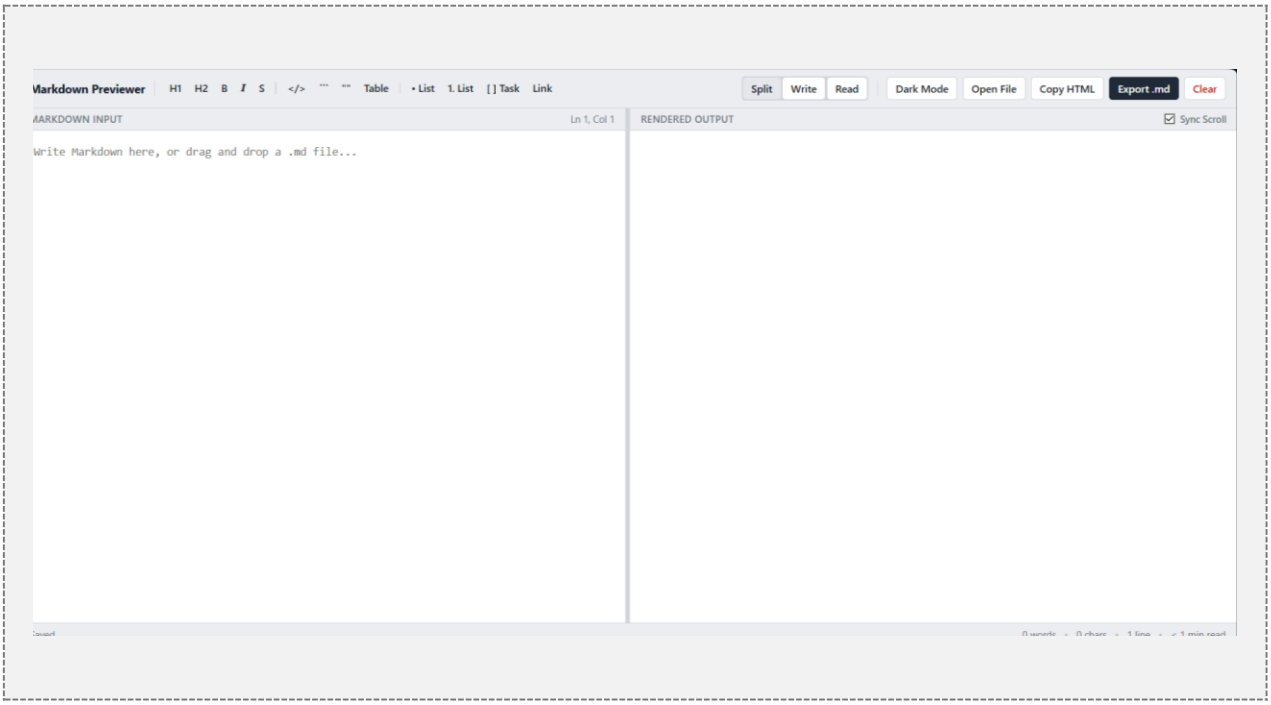


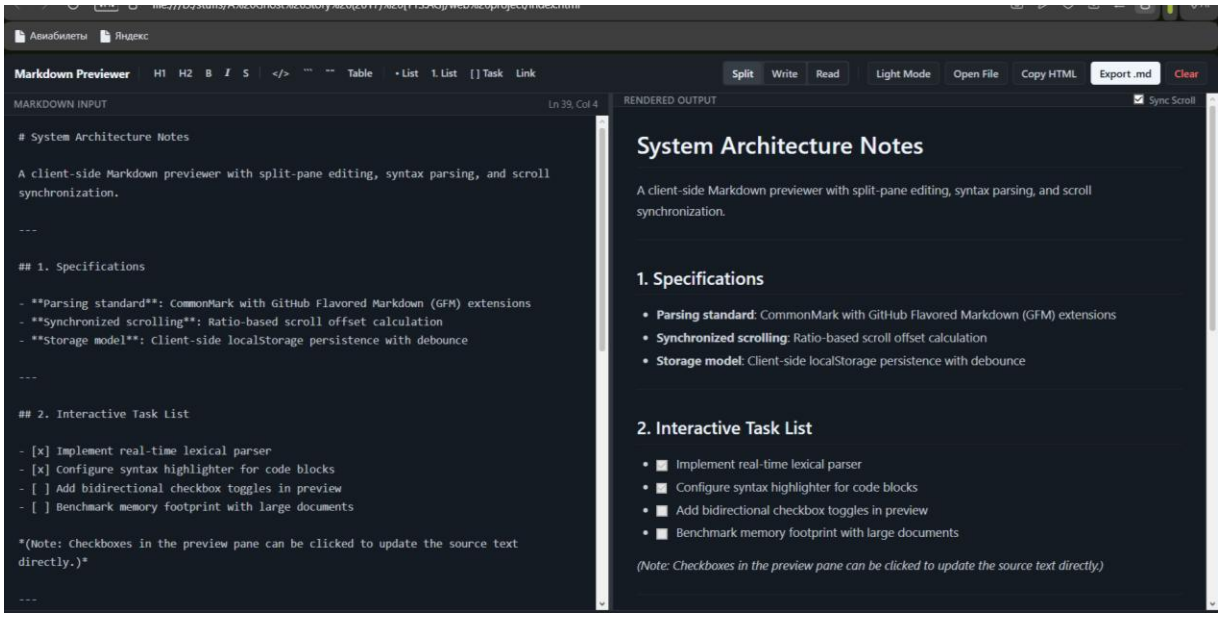
3. MODULE DESCRIPTION

- **Editor Module (editor.js):** Provides the writing experience inside the textarea: Tab and Shift+Tab indentation, smart list continuation, bracket and quote auto-pairing, pair backspacing and keyboard shortcuts for bold, italic, links and strikethrough.
- **Rendering Module (renderer.js):** Converts the Markdown text to HTML with marked.js, applies Highlight.js to code blocks, adds the language pill and Copy Code button, converts GitHub alert blockquotes into callouts and renders KaTeX formulas.
- **Toolbar Module (index.html and editor.js):** The toolbar buttons carry data-action attributes. Each action wraps the selected text or inserts a Markdown template (headings, lists, tables, code blocks, task items and links) at the cursor.
- **Application Controller (app.js):** The main controller that holds the application state and connects the editor, renderer and interface. It manages the view modes, the draggable resizer, synchronised scrolling, status bar statistics, theme switching, auto-save and file export.
- **View and Layout Module:** The Split, Write and Read buttons change the mode class of the workspace element. The resizer adjusts the pane widths between 15% and 85% and resets to a 50/50 balance on double-click.
- **Scroll Synchronisation:** Calculates the scroll ratio of the active pane and applies it proportionally to the other pane; the Sync Scroll checkbox enables or disables it.
- **File and Export Module:** Reads imported files with the FileReader API, downloads Markdown through a Blob, produces standalone HTML, copies rendered HTML with the Clipboard API and prints through a dedicated print stylesheet.
- **Storage and Theme Module:** Writes the document and the theme preference to localStorage, restores them on load and shows the save status in the footer.
- **Templates Module (templates.js):** Stores the preset documents (technical README, editorial essay, sprint sync and Markdown syntax guide) that can be loaded into the editor.
- **Stylesheets (style.css and markdown.css):** style.css defines the application layout, split-pane system, theme tokens and print rules; markdown.css defines the typography of the rendered output, including headings, tables with zebra striping, blockquotes, callouts and code blocks.
- **HTML Structure (index.html):** The semantic skeleton containing the header with the toolbar and view controls, the editor pane, the resizer, the preview pane, the status bar and the toast notification. The rendered Markdown is injected into #preview-output by JavaScript.

4. IMPLEMENTATION

4.1 SCREENSHOTS





4.2 DATA DESIGN

There is no database. The document is held in the editor textarea and the user preferences are stored in the browser's localStorage. The application keeps the following items:

Field	Type	Example
Markdown content	String	"# Hello Markdown"
Theme	String	"light"
View mode	String	"split"
Split ratio	Number	50
Sync scroll	Boolean	true
Save status	String	"Saved"
Word count	Integer	128
Character count	Integer	742
Line count	Integer	24
Reading time	String	"1 min read"

4.3 CODE

LIVE MARKDOWN RENDERING (renderer.js)

```
const input  = document.getElementById('markdown-input');
const output = document.getElementById('preview-output');

function renderMarkdown() {
  output.innerHTML = marked.parse(input.value);

  output.querySelectorAll('pre code').forEach(block => {
    hljs.highlightElement(block);
  });
  updateStats();
}

input.addEventListener('input', renderMarkdown);
```

TOOLBAR FORMATTING (editor.js)

```
function wrapSelection(before, after = before) {
  const start = input.selectionStart;
  const end   = input.selectionEnd;
  const text  = input.value.slice(start, end);
  input.setRangeText(before + text + after, start, end, 'select');
  input.focus();
  renderMarkdown();
}

const actions = {
  bold:   () => wrapSelection('**'),
  italic: () => wrapSelection('*'),
  strike: () => wrapSelection('~~'),
  code:   () => wrapSelection('`'),
  h1:     () => prefixLine('# '),
  h2:     () => prefixLine('## ')
};

document.querySelectorAll('.tool-btn').forEach(btn => {
  btn.addEventListener('click', () => actions[btn.dataset.action]?.());
});
```

VIEW MODE SWITCHING (app.js)

```
document.querySelectorAll('.btn-toggle').forEach(btn => {
  btn.addEventListener('click', () => {
    workspace.className = `workspace mode-${btn.dataset.view}`;
    document.querySelectorAll('.btn-toggle').forEach(b => {
      b.classList.toggle('active', b === btn);
    });
  });
});
```

AUTO-SAVE WITH LOCALSTORAGE (app.js)

```
let saveTimer;

input.addEventListener('input', () => {
  saveStatus.textContent = 'Saving...';
  clearTimeout(saveTimer);
  saveTimer = setTimeout(() => {
    localStorage.setItem('markdown-content', input.value);
    saveStatus.textContent = 'Saved';
  }, 400);
```

```
});
```

THEME TOGGLE WITH LOCALSTORAGE (app.js)

```
themeBtn.addEventListener('click', () => {
  const root = document.documentElement;
  const next = root.getAttribute('data-theme') === 'light' ? 'dark' : 'light';
  root.setAttribute('data-theme', next);
  localStorage.setItem('markdown-theme', next);
  themeBtn.textContent = next === 'light' ? 'Dark Mode' : 'Light Mode';
});
```

SPLIT-PANE LAYOUT (style.css)

```
.workspace { display: flex; height: calc(100vh - 96px); }
.mode-split .pane { flex: 1; }

.mode-edit .preview-pane,
.mode-edit .resizer { display: none; }

.mode-preview .editor-pane,
.mode-preview .resizer { display: none; }
```

5. CONCLUSION

Marksmith demonstrates that a feature-rich Markdown editing environment can be built with pure HTML, CSS and JavaScript, using only small, focused libraries for parsing and syntax highlighting and no frameworks or build tools. The project provided practical experience in real-time DOM updates, event-driven programming, text-selection handling in a textarea, split-pane layouts with a draggable divider, proportional scroll synchronisation, file import and export with browser APIs, and localStorage-based persistence. The writer-focused design also showed how clear typography, themes and a clean layout make a tool comfortable to use for long sessions. Future improvements could include multiple documents with tabs, cloud or file-system sync, a find-and-replace panel, diagram support such as Mermaid, collaborative editing and a PWA offline mode.

6. REFERENCES

- MDN Web Docs (HTML5, CSS Flexbox, JavaScript DOM, FileReader, Clipboard API, localStorage), developer.mozilla.org
- CommonMark Specification, commonmark.org
- GitHub Flavored Markdown Spec, github.github.com/gfm
- marked.js Documentation, marked.js.org
- Highlight.js Documentation, highlightjs.org
- KaTeX Documentation, katex.org
- Visual Studio Code Documentation, code.visualstudio.com
- WebAIM Accessibility Guidelines, webaim.org